

Računarska vežba 3

Python uslovni iskazi i iterativne petlje



1 Uslovni iskazi IF, ELIF, ELSE

1.1 Osnovna naredba IF:

Struktura jednograme naredbe **IF** prikazana je na slikama. Uslov između ključne reči **IF** i dvotačke (**:**) predstavlja logički izraz čija vrednost može biti istinita (**True**) ili lažna (**False**). Logički izrazi grade se najčešće uz pomoć relacija poređenja i logičkih operacija **AND**, **OR** i **NOT**.

Npr. izraz koji testira da li tačka (**x;y**) pripada prvom kvadrantu glasi: **x >= 0 and y >= 0**. Da bi izraz bio tačan (tačka pripada prvom kvadrantu), oba uslova povezana operacijom **and** moraju biti ispunjena. Ako je odgovarajući uslov naredbe **IF** ispunjen, tok programa prebacuje se na blok naredbi **blok_1**. Blok predstavlja uzastopni niz naredbi koje čine jednu smislenu programsku celinu. Program, kao jedinstvena celina, tretira se kao sveobuhvatni blok.

Sve naredbe bloka IF moraju biti uvučene udesno za jednak broj mesta u odnosu na početak reda u kome se nalazi IF. Prva naredba van bloka, `naredba_posle`, pomerena je sleva u odnosu na blok i poravnata je sa naredbom if. Ako uslov nije ispunjen, `blok_1` se preskače i izvršava se `naredba_posle`. Ova naredba izvršava se i kada je uslov ispunjen, ali pošto se prvo izvrše sve naredbe u bloku. Naglašava se da svaki blok naredbe IF može da sadrži proizvoljan broj umetnutih IF blokova. Na taj način tok programa nastavlja da se grana u zavisnosti od odgovarajućih logičkih uslova.



Primer:

```
>>> poeni=70
>>> if poeni >= 51:
...     print("Položen test")
...
...
Položen test
>>>
```

U ovom kodu definisana je promenljiva `poeni` sa celobrojnomo vrednošću 70. Zatim se u sledećem redu koristi IF iskaz da bi se odredilo da li je promenljiva `poeni >= 51`. Ako jeste, program treba da ispiše poruku "Položen test"

Primer:

```
1  # Primer 2: Ako je broj pozitivan, ispisati poruku
2
3  broj = 3
4  if (broj > 0): # ako je uslovni izraz TRUE, prelazi se na IF naredbu
5      print(broj, "je pozitivan broj.")
6
7  print("Sledeće linije koda...")
8
9  broj = -1
10 if broj > 0: # ako je uslovni izraz FALSE, ne prikazuje se tekst
11     print(broj, "je negativan broj.")
12
13 print("Sledeće linije koda opet....")
```

```
3 je pozitivan broj.
Sledeće linije koda...
Sledeće linije koda opet....
```

U ovom primeru, `broj > 0` is the test izraz. Naredba `if` se izvršava samo ako je test izraz tačan tj. ima logičku vrednost `True`. U liniji 3, pošto je promenljiva `broj=3`, test izraz je `True` i izvršava se komanda `print` u bloku za `IF` (linija 4). U drugom slučaju je `broj=-1` test izraz je `False`, pa se komanda `print` u bloku `IF` ne izvršava. Komande za `print()` nakon `IF` bloka naredbi, u oba slučaja, ne pripadaju samom `IF` bloku, tj. nisu uvučene, tako da se izvršavaju bez obzira na vrednost test izraza.

Primer:

```
1  # Primer 3: Računa se kvadrat broja ako je on veći od 6
2
3  broj = 9
4  if broj > 6:
5      # Računanje kvadrata
6      print(broj * broj)
7
8  print('Sledeće linije koda')
```

81

Sledeće linije koda

Primer:

```
1  if 5 != 3 * 6:
2      print ("Hello!")
```

Hello!

```
1  if 5 == 15/3:
2      print ("Hello!")
```

Hello!

Primer:

Komanda `IF` može se napisati samo u jednom redu, radi bolje preglednosti. Koristi se u slučaju da se izvršava samo jedna linija koda u `IF` bloku.

```
1  broj1, broj2 = 5, 6
2  if(broj1 < broj2): print("Broj1 je manji od Broj2")
```

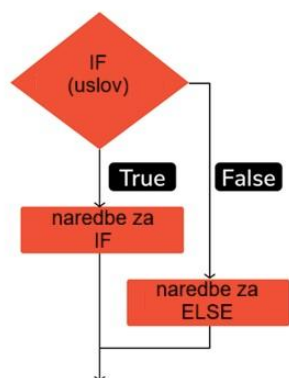
Broj1 je manji od Broj2

Primer:

```
1  #program za proveru da li je num1 manji od num2
2  num1, num2 = 6, 5
3  if (num1 < num2):
4      print("num1 je manji od num2")
```

U ovom slučaju Python IDE neće ispisati nikakav tekst!!!! Pošto je $6 > 5$, to znači da je uslovni iskaz `False`. Interpreteru je data instrukcija da komandom `print()` štampa poruku samo kada je uslovni iskaz `True`, tako da ovde neće biti prikazan izlaz. Za instrukciju šta će se dešavati u slučaju ako je uslovni iskaz `False`, koristi se ključna reč `ELSE`.

1.2 Uslovni iskaz IF-ELSE:



```
if uslov:
    blok_1
else:
    blok_2

naredba_posle
```

Primer:

```
1 # Primer 1:
2
3 poeni = 48
4
5 if poeni >= 51:
6     print("Položen ispit")
7 else:
8     print("Nepoložen ispit")
```

Nepoložen ispit

Pošto je navedeni broj poena 48 veći od 60, uslovni iskaz `poeni >= 51` će biti `False`, pa se neće štampati print poruka u iskazu `IF`, već će se naredbom `print` u okviru `else` ispisati tekst "Nepoložen ispit".

Primer:

```
1 # Program koji proverava da li je broj pozitivan ili negativan
2 # i prikazuje odgovarajuću poruku
3
4 broj = 3
5
6 # Uraditi za vrednosti
7 # broj = -5
8 # broj = 0
9
10 if broj >= 0:
11     print("Broj je pozitivan ili nula")
12 else:
13     print("Broj je negativan")
```

Broj je pozitivan ili nula

U ovom primeru kada je `broj=3`, uslovni iskaz je `True` pa se izvršava naredba `print()` u okviru `IF`, dok se naredba u okviru `else` preskače. Ako je `broj=-5`, uslovni iskaz je `False`, preskače se naredba u okviru `IF`, dok se naredba `print()` u okviru `else` izvršava. Ako je `broj=0`, program se izvršava kao u prvom slučaju.

Primer:

Ako postoji samo po jedan iskaz i u `IF` i u `ELSE`, kod se može pojednostaviti pisanjem u jednoj liniji.

```
1 broj1, broj2 = 6, 5
2 print("broj1 je manji od broj2") if (broj1 < broj2) else print("broj2 je manji od broj1")
```

broj2 je manji od broj1

```
1 number = 96
2 if number > 0: print("positive")
3 else: print("negative")
```

positive

Primer:

```
1 # Program proverava da li je broj1 manji od broj2
2
3 broj1, broj2 = 5, 5
4 if (broj1 < broj2):
5     print("Broj1 je manji od Broj2")
6 else:
7     print("Broj2 je manji od Broj1")
```

Pošto su obe vrednosti promenljivih iste (`broj1, broj2=5,5`), uslovni iskaz `broj1<broj2` je `False`, pa se kontrola prebacuje na `else` blok i `print()` metodu koja ispisuje se tekst u okviru nje. Međutim, ovo nije logički korektno jer su obe promenljive iste vrednosti (`5=5`). Potreban je dodatni uslovni iskaz kojim će se ispitati da li je `broj1=broj2`.

Primer:

Za igru par-nepar treba napisati kod koji prvo “pita” igrača za celobrojni ulog koji stavlja u opticaj. Potom računar zamišlja broj x , a igrač pogađa da li je x paran ili ne. Pogodak igraču donosi, a promašaj uzima novac u visini uloga. Zamišljanje slučajnog broja će biti improvizovano tako što će zamišljeni broj biti jednak unetoj visini uloga.

Operacija `%`, u logičkom izrazu naredbe `if`, daje ostatak pri celobrojnem deljenju. Dvograna naredba `if` podrazumeva postojanje bloka naredbe `else` koji se izvršava kada uslov nije ispunjen. Ako unos parnosti nije adekvatan (`r5`), izvršava se blok `else` u kome se ispisuje poruka upozorenja (`r11`). Za korektan unos

ispituje se parnosti i ispisuje ishod partije (r6-9). Tom prilikom koristi se unutrašnja if-else naredba koja je umetnuta u blok prve naredbe if.

```

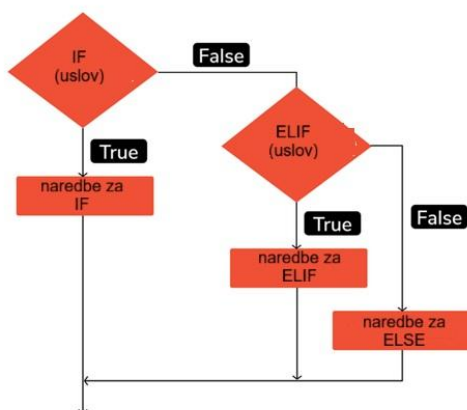
1  # igra par-nepar
2  ulog = int(input('koliki je ulog (ceo broj): '))
3  pokusaj = int(input('par (0) ili nepar (1) '))
4
5  if pokusaj == 0 or pokusaj == 1:
6      if pokusaj == ulog % 2:
7          print('zaradili ste', ulog)
8      else:
9          print('izgubili ste', ulog)
10 else:
11     print('za parnost se mora uneti 0 ili 1!')
```

```

koliki je ulog (ceo broj): 15
par (0) ili nepar (1) 0
izgubili ste 15
```

1.3 Uslovni iskaz IF-ELIF-ELSE:

Kada postoji potreba da se, u zavisnosti od složene kombinacije uslova, izvrši samo jedan od tri ili više mogućih blokova, koristi se višegrana naredba **if-elif-else**. **ELIF** je skraćenica od **ELSE IF** i omogućava proveru višestrukih izraza. Ako je **IF** uslovni iskaz **False**, proverava se **ELIF** uslovni iskaz itd... Ako su svi uslovni iskazi **False**, izvršavaju se naredbe u okviru **ELSE**. Samo jedan blok među nekoliko **if-elif-else** blokova se izvršava u skladu sa uslovom. **IF** blok naredbi može imati samo jedan **else** blok, ali može imati više **elif** blokova.



```

if uslov_1:
    blok_1
elif uslov_2:
    blok_2
elif uslov_3:
    .....
elif uslov_n-1:
    blok_n-1
else:
    blok_n

naredba_posle
```

Primer:

```
1  '''Program proverava da li je broj pozitivan, negativan
2  ili nula, i prikazuje odgovarajuću poruku'''
3
4  broj = -20
5
6  # Proveriti i za vrednosti:
7  # broj = 0
8  # broj = -4.5
9
10 if broj > 0:
11     print("Pozitivan broj")
12 elif broj == 0:
13     print("Nula")
14 else:
15     print("Negativan broj")
```

Negativan broj

Primer:

```
1  # Program ispituje da li su brojevi veći, manji ili jednaki
2
3  broj1, broj2 = 5, 5
4  if(broj1 > broj2):
5      print("broj1 je veći od broj2")
6  elif(broj1 == broj2):
7      print("broj1 je jednak broj2")
8  else:
9      print("broj1 je manji od broj2")
```

broj1 je jednak broj2

Primer:

```
1  # Program koji određuje ocenu na ispitu
2
3  poeni = 55
4
5  if poeni >= 91:
6      print("Ocena 10")
7  elif poeni >= 81:
8      print("Ocena 9")
9  elif poeni >= 71:
10     print("Ocena 8")
11 elif poeni >= 61:
12     print("Ocena 7")
13 elif poeni >= 51:
14     print("Ocena 6")
15 else:
16     print("Nije položen ispit")
```

Primer:

```

1  # ispisuje veci od dva uneta broja
2  # ako su brojevi jednaki ispisuje prigodnu poruku
3
4  x = float(input('x= '))
5  y = float(input('y= '))
6
7  if x > y:
8      print('veci broj je', x)
9  elif x < y:
10     print('veci broj je', y)
11 else:
12     print('brojevi su jednaki!')
```

```

x= 45
y= 34
veci broj je 45.0
```

1.4 Ugnježdeni iskaz IF:

Python podržava formu ugnježdenih `if-else` ili `if-elif-else` iskaza unutar drugog `if-else` iskaza. Ugnježdeni `if` iskaz je koristan kada treba doneti seriju odluka. Bilo koji broj ovih naredbi može biti ugnježđen jedan u drugi. Uvlačenje je jedini način da se utvrdi nivo ugnježđenja, ali ovakve forme su podložne greškama pa ih treba izbegavati osim ako nije neophodno. Možemo koristiti ugnježdene `if` naredbe za situacije kada želimo da proverimo sekundarni uslov ako se prvi uslov izvrši kao Tačno.

Sintaksa:

<pre> if conditon_outer: if condition_inner: statement of nested if else: statement of nested if else: statement ot outer if else: Outer else statement outside if block</pre>	<pre> if expression1: statement(s) if expression2: statement(s) elif expression3: statement(s) elif expression4: statement(s) else: statement(s) else: statement(s)</pre>
--	---

Primer:

```
1  # Primer ugneženih IF...
2
3  a=10
4  if a>=20: # Uslov je FALSE
5      print ("Uslov je TRUE")
6  else: # Kod prelazi u ELSE blok
7      if a>=15: # Uslov FALSE
8          print ("Proveri drugu vrednost")
9      else: # Kod odlazi na ELSE blok
10         print ("Svi uslovi su FALSE")
11
```

Svi uslovi su FALSE

Primer:

```
1  # Zadatak koji određuje da li je broj pozitivan i paran
2
3  a = -4
4  if a > 0:
5      if a % 2 == 0:
6          print('A je pozitivan i paran broj')
7      else:
8          print('A je pozitivan broj')
9  elif a == 0:
10     print('A je nula')
11 else:
12     print('A je negativan broj')
13
```

A je pozitivan i paran_broj

Primer:

Ugnežđena IF petlja se može uprostiti upotrebom logičkog operatora AND.

```
1  # Zadatak koji određuje da li je broj pozitivan i paran
2  # upotrebom logičkog operatora AND
3
4  a = 23
5  if a > 0 and a % 2 == 0:
6      print('A je pozitivan i paran broj')
7  elif a > 0 and a % 2 != 0:
8      print('A je pozitivan broj')
9  elif a==0:
10     print('A je nula')
11 else:
12     print('A je negativan broj')
13
```

A je pozitivan broj

Ugneždjena IF petlja se može uprostiti upotrebom logičkog operatora OR.

```

1  # Zadatak: Primer if-else petlje
2  # upotrebom logičkog operatora OR
3
4  user = 'Petar'
5  access_level = 3
6  if user == 'admin' or access_level >= 4:
7      print('Pristup odobren!')
8  else:
9      print('Pristup zabranjen!')
```

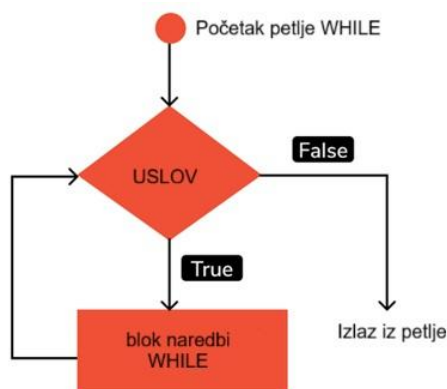
Pristup zabranjen!

2 Iterativne petlje WHILE i FOR

2.1 Petlja WHILE:

Naredba **WHILE** omogućava da se odgovarajući programski blok ponavlja u petlji sve dok je ispunjen uslov za izvršavanje petlje koji se navodi neposredno iza WHILE. Svaki prolaz kroz naredbe bloka petlje zove se iteracija. Program može da se zadrži u petlji beskonačno dugo ako je uslov za izvršavanje uvek ispunjen. Zbog toga je potrebno obezbediti da se ispunjenost uslova može menjati unutar bloka petlje WHILE. Petlja se uopšte ne izvršava samo ako uslov nije ispunjen pri prvom testiranju.

Prilikom rešavanja raznorodnih problema postoji potreba da se određeni algoritamski koraci ponove unapred poznati broj puta. Tada se koriste brojačke petlje koje uvode pojam **brojča**. Brojač je promenljiva koja ukazuje na to koliko je puta obavljen željeni blok operacija. U svakoj iteraciji petlje brojčac se obično uvećava za jedan, a petlja se završava kada je dostignut potreban broj ponavljanja.



Primer:

```
1  # Primer: Ispis brojeva manjih od 5
2
3  brojac = 1
4  # petlja se izvršava sve dok je brojac manji od 5
5  while brojac < 5:
6      print(brojac)
7      brojac = brojac + 1
```

```
1
2
3
4
```

Primer:

```
1  # Primer: Program daje zbir brojeva do zadatog broja
2
3  broj = 10
4  zbir = 0
5  i = 1
6  while i <= broj:
7      zbir = zbir + i
8      i = i + 1
9  print("Zbir prvih 10 brojeva je:", zbir)
```

Zbir prvih 10 brojeva je: 55

Primer:

```
1  # Primer: Petlja WHILE
2
3  a=10      # 'a' je promenljiva
4
5  while a>0: # Uneti blok naredbi WHILE petlje jer je uslov TRUE
6      print ("Vrednost a je",a)
7      a=a-2
8  print ("WHILE petlja je završena")
```

Vrednost a je 10
Vrednost a je 8
Vrednost a je 6
Vrednost a je 4
Vrednost a je 2
WHILE petlja je završena

Primer:

```
1  # Primer: Koliko iteracija je potrebno,
2  # kada se broj deli sa 3 pre nego što bude <= 10
3
4  iter = 0
5  broj = 2500
6  while broj > 10:
7      # broj se deli sa 3
8      broj = broj / 3
9      # iteracija se povećava za 1
10     iter = iter + 1
11     print('Ukupni broj iteracija', iter)
12
```

Ukupni broj iteracija 6

Primer:

```
1  # Primer: Program koji sabira brojeve do zadatog broja zbir = 1+2+3+...+n
2
3  # Modifikovati program da korisnik unese broj
4  # n = int(input("Unesi broj n: "))
5
6  n = 100
7
8  # inicijalizacija zbira i brojača
9  zbir = 0
10 i = 1
11
12 while i <= n:
13     zbir = zbir + i
14     i = i+1    # update brojača, menja se redom 1, 2...
15
16 # print zbira
17 print("Zbir brojeva je", zbir)
```

Zbir brojeva je 5050

Primer:

```
1  # Program koji ispisuje unetu poruku n puta
2
3  poruka = input('Vaša poruka je: ')
4  n = int(input('Koliko puta da se ispiše? '))
5
6  i = 0 # inicijalizacija
7  while i < n: # da li je kraj
8      print(i+1, poruka) # koraci koji se ponavljaju n puta
9      i = i + 1 # uvećanje brojača
10
11     print('Da li je ispis OK? ')
```

```
Vaša poruka je: Ovo je Python predmet
Koliko puta da se ispiše? 5
1 Ovo je Python predmet
2 Ovo je Python predmet
3 Ovo je Python predmet
4 Ovo je Python predmet
5 Ovo je Python predmet
Da li je ispis OK? _
```

Primer:

WHILE petlja može imati opcioni **if-else** blok koji se koristi kada je potrebna uslovna iteracija. Ako je uslov **True**, onda će se naredbe unutar **IF** bloka izvršiti, u suprotnom, izvršiće se naredbe u okviru **ELSE** bloka.

```
1  # Primer: Ispis parnih i neparnih brojeva
2  # između 1 i zadatog broja
3
4  n = int(input('Uneti broj: '))
5  while n > 0:
6      # proverava da li je paran ili neparan
7      if n % 2 == 0:
8          print(n, 'je paran broj.')
9      else:
10         print(n, 'je neparan broj.')
11         # smanjujemo broj za 1 u svakoj iteraciji
12         n = n - 1
```

```
Uneti broj: 9
9 je neparan broj.
8 je paran broj.
7 je neparan broj.
6 je paran broj.
5 je neparan broj.
4 je paran broj.
3 je neparan broj.
2 je paran broj.
1 je neparan broj.
```

Primer:

```
1  # Primer: Ispis svih parnih i kvadrata neparnih
2  # brojeva manjih od zadatog broja n
3
4  n = 10
5  i = 1
6  while i < n:
7      #if (i % 2 == 0): (za parne brojeve, ostatak=0?)
8      if (i % 2 != 0):
9          print(i ** 2)
10         i = i + 1
11     else:
12         print(i)
13         i = i + 1
14
```

```
1
2
9
4
25
6
49
8
81
```

Primer:

WHILE petlja može imati opcioni **ELSE** blok. Naredbe u bloku **ELSE** se izvršavaju ako uslov u **WHILE** petlji dobije vrednost **False**. Blok naredbi u **ELSE** će biti preskočen ako se **WHILE** petlja naglo prekine ili se koristi **BREAK** iskaz da bi se **WHILE** petlja prekinula.

```
1 # Primer: Upotreba while loop za prikaz brojeva 1 to 10
2
3 i = 1
4 while i <= 7:
5     print(i)
6     i = i + 1
7 else:
8     print("OK. 'while loop' je izvršena korektno")
9
```

```
1
2
3
4
5
6
7
OK. 'while loop' je izvršena korektno
```

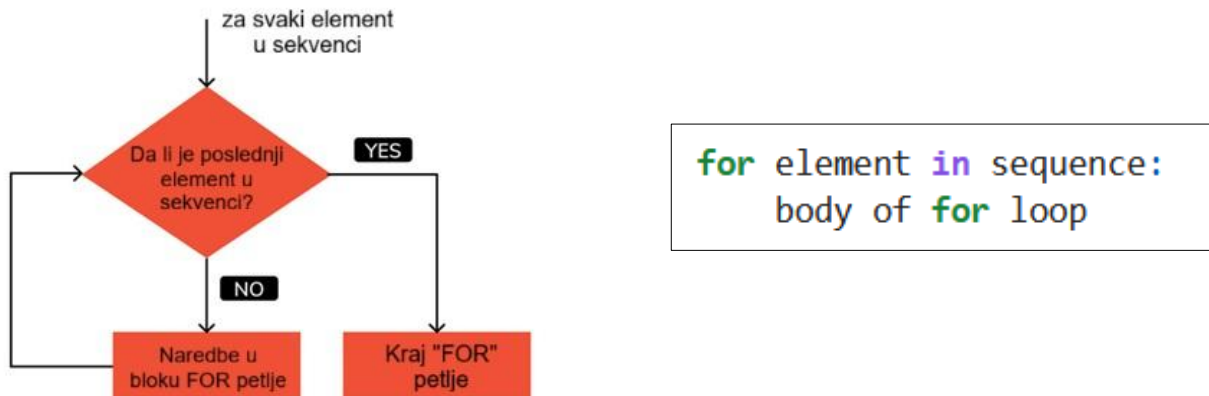
Primer:

```
1 # Primer: Provera da li je korisnik uneo broj između 100 i 600
2
3 broj = int(input('Unesi broj između 100 i 600: '))
4 # broj veći od 100 i manji od 600
5 while broj < 100 or broj > 600:
6     print('Broj nije u opsegu, treba ponovo uneti broj!')
7     broj = int(input('Ponovo unesi broj između 100 i 600: '))
8 else:
9     print("Unet je ispravan broj", broj)
10
```

```
Unesi broj između 100 i 600: 750
Broj nije u opsegu, treba ponovo uneti broj!
Ponovo unesi broj između 100 i 600: 150
Unet je ispravan broj 150
```

2.2 Petlja FOR:

FOR petlja omogućava ponavljanje sekvenci naredbi. Ova vrsta petlje vrši iteracije na osnovu vrednosti koje uzima iz lista (nizova), stringova i rečnika. Ovaj način iteracije čini for petlju pogodnijom za onaj deo koda koji želimo da se dogodi određen broj puta.



Sintaksa za iterativnu petlju FOR:

Element je promenljiva koja uzima vrednost unutar sekvence u svakoj iteraciji. Svi iskazi u telu FOR petlje se izvršavaju sa istom vrednošću. Blok FOR petlje je odvojen od ostatka koda pomoću uvlačenja. Petlja se nastavlja dok se ne dođe do poslednjeg elementa u sekvenci.

Primena FOR petlje je višestruka:

- Konačan broj iteracija: Ako se zna koliko puta treba izvršiti petlju, tada se koristi FOR petlja
- Smanjuje složenost koda: Petlja ponavlja određeni blok koda fiksni broj puta i smanjuje ponavljanje linija koda. Korišćenjem FOR petlje i WHILE petlje mogu se automatizovati i ponavljati zadaci na efikasan način
- Petlja kroz sekvence: Koristi se za iteraciju kroz liste, stringove, torke, rečnike itd. i izvršavanje različitih operacija nad njima, na osnovu uslova koji se prethodno definišu

Primer:

```

1  # Primer: For petlja
2
3  brojevi = ['one', 'two', 'three', 'four', 'five']
4
5  for i in brojevi:
6      print(i)
7

```

```

one
two
three
four
five

```

Primer:

```

1  # Primer: Srednja vrednost liste brojeva
2
3  brojevi = [10, 20, 30, 40, 50]
4
5  # konačna iteracija
6  # 5 iteracija petlje jer ima 5 elemenata liste
7
8  zbir = 0
9  for i in brojevi:
10     zbir = zbir + i
11  list_size = len(brojevi)
12  srednja_vr = zbir / list_size
13  print(srednja_vr)
14

```

```

30.0

```

Petlja FOR sa funkcijom `range()`:

Funkcija `range()` generiše sekvencu celih brojeva. Funkcija je zapravo konstruktor jer se njome kreira objekat klase `range` koji obuhvata više celobrojnih objekata.

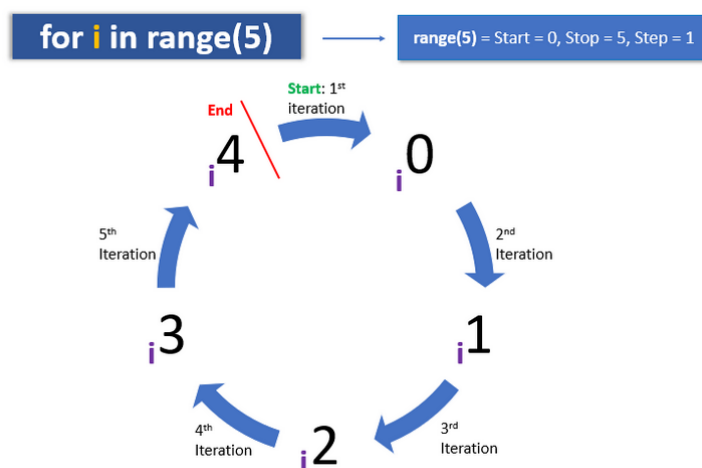
```

for i in range(n):      # i = 0,1,2, ..., n-1
    print(i+1, poruka)  # koraci koji se ponavljaju n puta

```

Naredba `for` u prethodnom kodu se čita na sledeći način: Za svako `i` iz sekvence od `0` do `n-1`, uradi sledeće: ...". Promenljiva `i` u svakoj iteraciji petlje, ukazuje na jedan celobrojni objekat čija je vrednost u rasponu od `0` do `n-1`. Ako su argumenti za `range()` takvi da je sekvenca prazna, brojačka promenljiva ne ukazuje ni na jednu konkretnu vrednost, pa se blok petlje ne izvršava. Brojačka promenljiva se koristi za različita izračunavanja u bloku petlje.

Funkcija `range()` vraća niz brojeva počevši od 0 (podrazumevano) ako početna granica nije navedena i povećava se za 1 (podrazumevano) dok se ne dostigne konačna granica. Funkcija `range()` se koristi za određivanje opsega (koliko puta) će se blok koda izvršiti. U navedenom primeru, generiše se niz brojeva upotrebom funkcije `range()`, pri čemu će `range(5)` generisati brojeve od 0 do 4 (5 brojeva)



Funkcija `range()` određuje koliko će puta biti ponovljena petlja, tako da formira niz brojeva od 0 do broja u argumentu umanjenog za 1. Može se zadati i proizvoljna donja granica.

Npr. `range(1, 5)` generiše niz brojeva `1, 2, 3, 4`.

Da su bili potrebni brojevi od 1 do 5, funkcija bi bila `range(1, 6)`

Brojevi koje generiše `range()` ne moraju biti uzastopni celi brojevi. Trećim argumentom funkcije može se zadati korak opsega.

Npr. `range(1, 10, 2)` daje brojeve od 1 do 9 sa korakom 2, odnosno `1, 3, 5, 7, 9`.

Brojevi mogu ići i unazad, tako da npr. `range(5, 1, -1)` daje brojeve od 5 do 2, unazad (korak -1), odnosno `5, 4, 3, 2`.

Primeri:

<code>range(10)</code>	<code>0, 1, 2, 3, 4, 5, 6, 7, 8, 9</code>
<code>range(1, 10)</code>	<code>1, 2, 3, 4, 5, 6, 7, 8, 9</code>
<code>range(3, 7)</code>	<code>3, 4, 5, 6</code>
<code>range(2, 15, 3)</code>	<code>2, 5, 8, 11, 14</code>
<code>range(9, 2, -1)</code>	<code>9, 8, 7, 6, 5, 4, 3</code>

Primer:

```
1  # Primer: Upotreba range funkcije u FOR?
2
3  # prazan opseg
4  print(list(range(0)))
5
6  # upotreba range(stop)
7  print(list(range(10)))
8
9  # upotreba range(start, stop)
10 print(list(range(1, 10)))
11 |
```

```
[]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Primer:

```
1  # Primer: Range funkcija sa start, stop i korakom
2
3  for i in range (2, 12, 2): # početak od 2, korak 2, kraj pre 12
4      print (i)
5
6
7
8
9
10
```

Primer:

```
1  # Primer: Naći zbir 10 brojeva preko FOR
2
3  zbir=0
4  for n in range(1,11): # range (1,11) znači (1,2,3,4,5,6,7,8,9,10)
5      zbir+=n
6      print (zbir)
7
8  '''
9  0+1 = 1
10 1+2 = 3
11 3+3 = 6
12 6+4 = 10
13 10+5 =15
14 21
15 28
16 36
17 45
18 45+10 = 55
19 '''
```

1
3
6
10
15
21
28
36
45
55

Primer:

```
1 # Primer: printing serije brojeva preko for i range
2
3 print("Sličaj 1:")
4 for i in range(5): # Ispis brojeva od 0 do 4
5     print (i)
6
7 print("Slučaj 2:")
8 for i in range(5, 10): # Ispis brojeva od 5 do 9
9     print (i)
10
11 print("Slučaj 3:")
12 for i in range(5, 10, 2): # Ispis od 5 do 9 sa korakom 2
13     print (i)
14
```

Sličaj 1:

0
1
2
3
4

Slučaj 2:

5
6
7
8
9

Slučaj 3:

5
7
9

Petlja FOR sa **if-else** i **else**:

FOR petlja može imati opcioni **if-else** blok. Blok **if-else** proverava uslov i ako je on True, izvršava se blok koda koji se nalazi unutar **if** bloka, a ako je uslov netačan, izvršava se blok koda koji se nalazi unutar **else** bloka.

Primer:

```
1 # Primer: Parni i neparni brojevi preko for i if-else
2
3 for i in range(1, 11):
4     if i % 2 == 0:
5         print('Paran broj:', i)
6     else:
7         print('Neparan broj:', i)
8
```

Neparan broj: 1
Paran broj: 2
Neparan broj: 3
Paran broj: 4
Neparan broj: 5
Paran broj: 6
Neparan broj: 7
Paran broj: 8
Neparan broj: 9
Paran broj: 10

Primer:

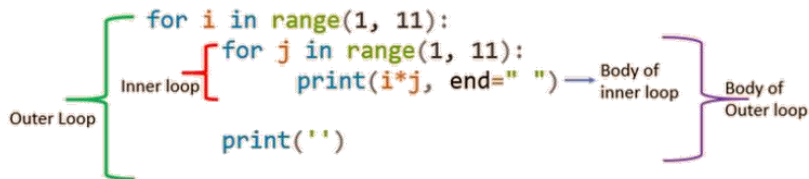
```
1 # Primer: Prikaz brojeva preko for i else
2
3 for broj in range(11):
4     print(broj) # štampa od 0 do 10, ne i 11
5 else:
6     print('Petlja se zaustavlja na:', broj)
7
```

0
1
2
3
4
5
6
7
8
9
10
Petlja se zaustavlja na: 10

Ugneždjena petlja FOR:

```
# outer for loop
for element in sequence
    # inner for loop
    for element in sequence:
        body of inner for loop
    body of outer for loop
other statements
```

Ugnežđena `for` petlja predstavlja petlju `for` unutar druge `for` petlje. U Pythonu se može koristiti bilo koja petlja unutar bilo koje druge petlje, npr. `for` petlja unutar `while` petlje, `while` unutar `for` in itd. Uglavnom se koristi kod rada sa dvodimenzionalnim nizovima. Kod ugnežđenih petlji, unutrašnja petlja (inner loop) mora da završi sve iteracije za svaku iteraciju spoljašnje petlje (outer loop). Za svaku iteraciju spoljašnje petlje, unutrašnja petlja se ponovo pokreće i završava sve iteracije, a zatim počinje sledeća iteracija spoljašnje petlje.



Spoljašnja `for` petlja koristi funkciju `range()` za iteraciju kroz prvih deset brojeva. Unutrašnja `for` petlja će se izvršiti deset puta za svaki prolaz kroz spoljašnju petlju. U telu unutrašnje petlje, ispisuje se naredba za množenje spoljašnjeg indeksa `i` i trenutnog indeksa `j` unutrašnje petlje. Unutrašnja petlja je u okviru bloka naredbispoljašnje petlje.

Primer:

```

1  # Primer: Ispis tabele množenja prvih 10 brojeva
2
3  # spoljašnja petlja
4  for i in range(1, 11):
5      # ugnežđena petlja
6      for j in range(1, 11):  # iteracija od 1 do 10
7          print(i * j, end=' ')  # print množenja i*j
8      print()
9
  
```

```

1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
  
```

Primer:

```

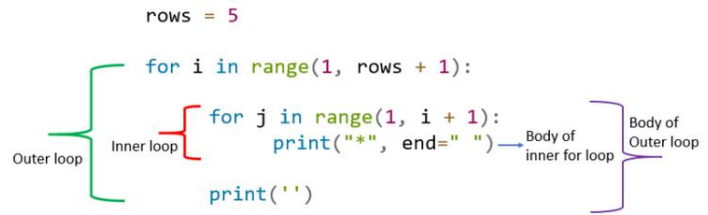
1 rows = 5
2 # outer loop
3 for i in range(1, rows + 1):
4     # inner loop
5     for j in range(1, i + 1):
6         print("*", end=" ")
7     print('\n')
8

```

```

*
* *
* * *
* * * *
* * * * *

```



Primer:

```

1 # Primer: Metood 1
2
3 for i in range(1,6):      # brojevi 0,1,2,3,4,5
4     for j in range(1, i+1):
5         print(i, end=" ") # print broj
6     print(" ")           # ispis u korektnom rasporedu
7

```

```

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

```

1 # Primer: Metood 2
2
3 rows = 6
4 for num in range(rows):      # opseg 0,1,2,3,4,5
5     for i in range(num):
6         print(num,end=" ") # print broja
7     print(" ")             # linija nakon svakog reda
8

```

Primer:

```

1 # Primer: Prikaz obrnuta piramida
2
3
4 for i in range (1,6):      # redovi od 1 do 5
5     for j in range (5,i-1,-1): # opseg kolona (5,0,-1) = 54321
6         print ("*", end=" "),
7     print (" ")
8

```

```

* * * * *
* * * *
* * *
* *
*

```

Primer:

```
1 # Primer: Ispis brojeva u redovima
2
3 for i in range(1,6):
4     for j in range(6-i):
5         print(j+1, end=" ")
6     print(" ")
7
8 """
9 i = 1 2 3 4 5
10 # loop 1
11 for i = 1, range (5): j=0 1 2 3 4
12 i = 1, print: 1 2 3 4 5
13 # loop 2
14 for i = 2, range (4): j=0 1 2 3
15 i = 2, print: 1 2 3 4
16 # loop 3
17 for i = 3, range (3): j=0 1 2
18 i = 3, print: 1 2 3
19 # loop 4
20 for i = 4, range (2): j=0 1
21 i = 4, print: 1 2
22 # loop 5
23 for i = 5, range (1): j=0
24 i = 5, print: 1
25 """
26
```

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Može se koristiti ugrađena funkcija `range()` sa `for` petljom da bi se obrnuo redosled elemenata. Funkcija `range()` generiše cele brojeve između datog početnog i završnog broja.

Primer:

```
1 # Primer: reverse for loop preko range()
2
3 print("Reverse numbers using for loop")
4 num = 5
5 # start = 5
6 # stop = -1
7 # step = -1
8 for num in (range(num, -1, -1)):
9     print(num)
10
```

```
Reverse numbers using for loop
5
4
3
2
1
0
```

Primer:

```

1  # Primer: reverse for - reversed, enumerate, zip
2
3  print("reversed: \t",end="")
4  for ch in reversed("abc"):
5      print(ch,end=";")
6
7  print("\nenumerated:\t",end="")
8  for i,ch in enumerate("abc"):
9      print(i,"=",ch,end="; ")
10
11 print("\nzip'ed: ")
12 for a,x in zip("abc","xyz"):
13     print(a,":",x)

```

```

reversed:      c;b;a;
enumerated:    0 = a; 1 = b; 2 = c;
zip'ed:
a : x
b : y
c : z

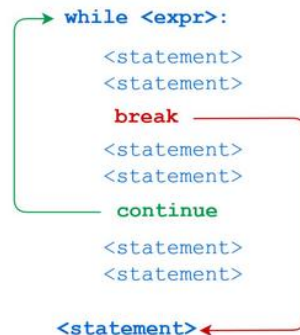
```

3 Naredbe BREAK, CONTINUE, PASS

Petlje u Pythonu se koriste za ponavljanje iteracija kroz blok koda. Ponekad je potrebno da se kontrola prebaci na drugo mesto u kodu kada se zadovolji određeni uslov. Tu dolaze do izražaja kontrolne naredbe. Kontrolne naredbe se koriste za kontrolu toka izvršavanja programa na osnovu navedenih uslova.

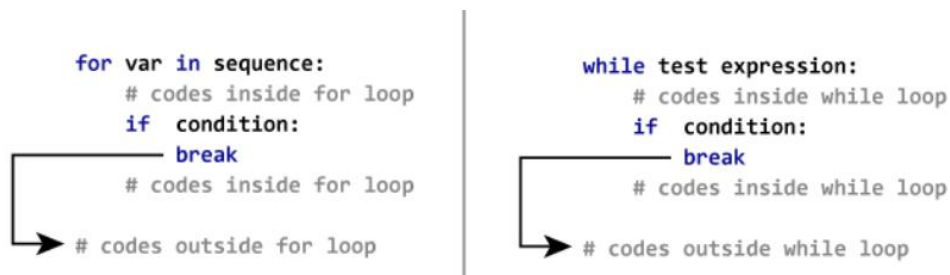
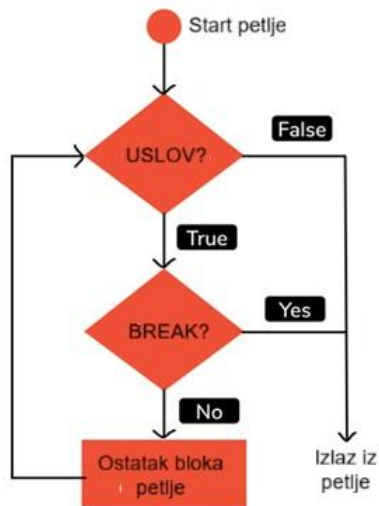
Python podržava 3 vrste kontrolnih naredbi:

Izraz	Opis
1 break	Prekida tekuću petlju, koristi se break za trenutni izlazak iz petlje
2 continue	Prekid trenutne iteracije petlje i prelaz na sledeću iteraciju
3 pass	Nastavak sa standardnim pokretanjem programa



Naredba BREAK:

Naredba **break** pruža mogućnost izlaska iz petlje kada se pokrene spoljašnji uslov, a kontrola programa se prenosi na sledeću naredbu koja sledi petlju. Ako se naredba **break** nalazi unutar ugneždene petlje (petlja unutar druge petlje), naredba **break** će prekinuti ugneždenu petlju. Naredbu **break** ćete postaviti unutar bloka koda ispod naredbe petlje, obično nakon uslovne naredbe **if**. Ova naredba se može koristiti samo u petljama **for** i **while**.



Način implementacije **break** iskaza u petljama **for** i **while**

Primer:

```

1  # Primer: primena break u for petlji
2
3  number = 0
4  for number in range(10):
5      number = number + 1
6      if number == 5:
7          break    # ovde je prekid
8      print('Broj je ' + str(number))
9  print('Van petlje!')
10
    
```

```
Broj je 1
Broj je 2
Broj je 3
Broj je 4
Van petlje!
```

U ovom programu, promenljiva `number` je inicijalizovana na 0. `For` naredbom formira se petlja sve dok je promenljiva `number` manja od 10. Unutar `for` petlje, `number` se postepeno povećava za 1 sa svakim prolazom zbog `number = number + 1`. `If` naredba predstavlja uslov da ako je promenljiva `number` ekvivalentna celom broju 5, petlja će se prekinuti. Unutar petlje se takođe nalazi `print()` naredba koja će se izvršavati sa svakom iteracijom `for` petlje dok se petlja ne prekine, pošto se nalazi posle `break` naredbe. Da bi se utvrdilo kada se dolazi van petlje, uneta je poslednja `print()` naredba izvan `for` petlje. Kada se dostigne uslov da je `number==5`, petlja se prekida pomoću `break` naredbe.

Primer:

```
1  # Primer: program proverava da li je 'A' u unetoj reči
2
3  a = input ("Unesi reč: ")
4  for i in a:
5      if (i == 'A'):
6          print ("A je pronađeno")
7          break
8  else:
9      print ("A nije pronađeno")
10
11 # Note: Python je case-sensitive jezik.
12
```

```
Unesi reč: Abrakadabra
A je pronađeno      _
```

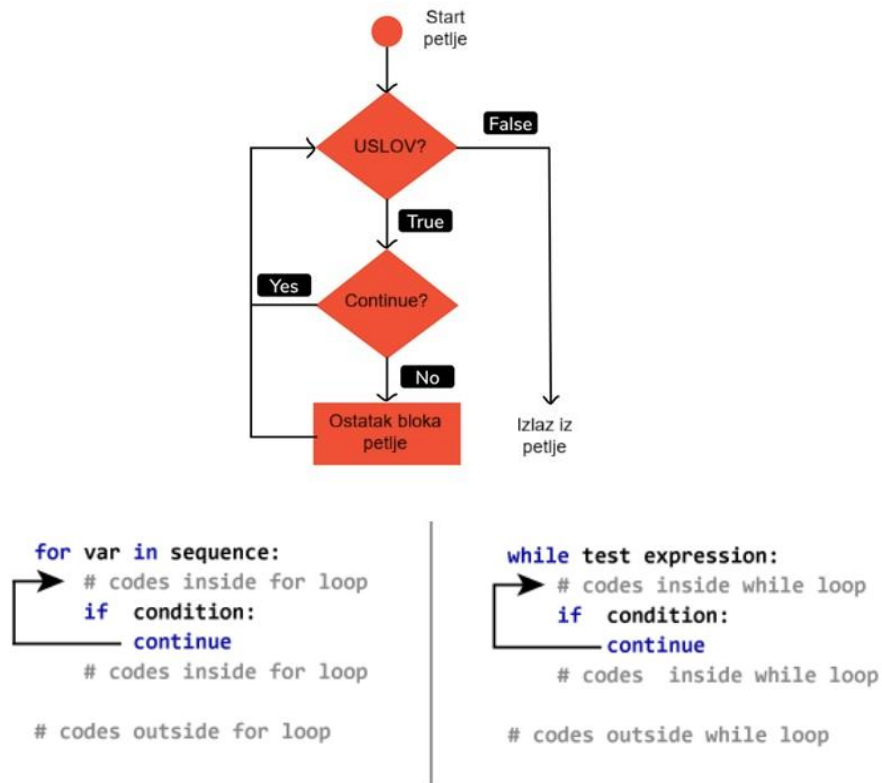
Primer:

```
1  # Primer: `while` petlja sa `break`
2  # Program štampa samo prvu reč uz unos space između slova
3
4  name = 'Petar99 Jana22 Marko55 '
5
6  size = len(name)
7  i = 0
8  # iteracija kroz petlju do zadnjeg karaktera
9  while i < size:
10     # break petlje ako je tekući karakter space
11     if name[i].isspace():
12         break
13     # print tekućeg karaktera
14     print(name[i], end=' ')
15     i = i + 1
16
```

```
P e t a r 9 9
```

Naredba CONTINUE:

Naredba `continue` je naredba za skok koja se koristi za preskakanje izvršavanja trenutne iteracije. Nakon preskakanja, petlja nastavlja sa sledećom iteracijom. Naredbu `continue` možemo koristiti samo sa petljama `for` i `while`. Obično se koristi samo unutar naredbe `if` (inače se ostatak petlje nikada ne bi izvršio)



Primer:

```

1  # Primer: upotreba `continue` unutar `while` petlje
2
3  ime = 'Aleksandra'
4
5  size = len(ime)
6  i = -1
7  # iteracija do zadnjeg elementa petlje
8  while i < size - 1:
9      i = i + 1
10     # preskače se blok petlje ako je tekući karakter space
11     if ime[i].isspace():
12         continue
13     # print tekućeg karaktera
14     print(ime[i], end=' ')
15
    
```

A l e k s a n d r a

Primer:

```

1  # Primer: `continue` iskaz u nested petlji
2
3  for i in range(1, 11):
4      print('Tabela množenja za', i)
5      for j in range(1, 11):
6          # uslov za prekid tekuće iteracije
7          if j == 5:
8              continue
9          print(i * j, end=' ')
10     print('')
11

```

```

Tabela množenja za 1
1 2 3 4 6 7 8 9 10
Tabela množenja za 2
2 4 6 8 12 14 16 18 20
Tabela množenja za 3
3 6 9 12 18 21 24 27 30
Tabela množenja za 4
4 8 12 16 24 28 32 36 40
Tabela množenja za 5
5 10 15 20 30 35 40 45 50
Tabela množenja za 6
6 12 18 24 36 42 48 54 60
Tabela množenja za 7
7 14 21 28 42 49 56 63 70
Tabela množenja za 8
8 16 24 32 48 56 64 72 80
Tabela množenja za 9
9 18 27 36 54 63 72 81 90
Tabela množenja za 10
10 20 30 40 60 70 80 90 100

```

Naredba PASS:

Ključna reč **pass** se koristi za izvršavanje „ničega“, što znači da kada ne želimo da izvršimo kod, **pass** se može koristiti za izvršavanje „praznih“ kodova. Ako se želi zaobići bilo koji kod, može se koristiti **pass** naredba, i ujedno se koristi kao rezervno mesto za buduće implementacije funkcija, petlji itd. Ako pretpostavimo da imamo petlju ili funkciju koja još nije implementirana, i želimo da je implementiramo u budućnosti koristimo **pass** naredbu.

Primer:

```
1  # Primer: Pass primena
2
3  broj = 0
4
5  for broj in range(10):
6      broj = broj + 1
7      if broj == 5:
8          pass # pass ovde
9      print('Broj je ' + str(broj))
10 print('Van petlje!')
11
```

```
Broj je 1
Broj je 2
Broj je 3
Broj je 4
Broj je 5
Broj je 6
Broj je 7
Broj je 8
Broj je 9
Broj je 10
Van petlje!
```

Korišćenjem `pass` naredbe u ovom programu, evidentno je da se program izvršava tačno onako kako bi se izvršavao da u programu nije bilo uslovne naredbe. `Pass` naredba govori programu da zanemari taj uslov i nastavi da izvršava program kao i obično.